

Quantum Algorithms for Finite Solvable Groups

Hector Li

December 12th, 2019

Solvable Groups I

Definition (Subnormal Series)

A subnormal series is a subgroup series where each member of the series is normal in the next one containing it. For example $H_0 \triangleleft H_1 \triangleleft \cdots \triangleleft H_n = G$ is a subnormal series of group G .

Definition (Normal Series)

A subnormal series is a normal series if every member of the series is a normal subgroup of the whole group.

Definition (Composition Series)

A composition series of a group G is a subnormal series of finite length $1 = H_0 \triangleleft H_1 \triangleleft \cdots \triangleleft H_n = G$, where H_{i-1} is the maximal (proper) subgroup of H_i where $i = 1, \dots, n$.

Solvable Groups II

Remark. The quotient group H_i/H_{i-1} is simple (its normal subgroups are only the trivial group and itself) for $i = 1, \dots, n$. If G is finite, then the quotient group H_i/H_{i-1} is cyclic of prime order.

Definition (Commutator)

For g, h in a group G , the commutator is defined as $[g, h] = g^{-1}h^{-1}gh$. For groups G, H , the commutator $[G, H]$ is defined as the group generated by all $[g, h]$ where $g \in G$ and $h \in H$. The derived subgroup of G is $G' = [G, G]$, and we write $G^{(0)} = G, G^{(1)} = G', \dots, G^{(n)} = G^{(n-1)'}.$ It is easy to verify that the derived subgroup is normal.

Definition (Derived Series (i.e. Commutator Series))

The derived series of a group G is the subnormal series $G \triangleright G' \triangleright G^{(2)} \triangleright \dots$.

Solvable Groups III

Definition (Solvable Groups)

The following definitions are equivalent. A group is solvable if

- There is a normal series of finite length starting from the trivial subgroup and ending at the whole group with each successive quotient group being an abelian group.
- there is a subnormal series of finite length starting from the trivial subgroup and ending at the whole group with each successive quotient group being an abelian group.
- the derived series reaches the identity in finitely many steps.

Remark. By considering the composition series, we can prove that a group G is solvable if and only if there exist elements $g_1, \dots, g_m \in G$ such that if we define $H_j = \langle g_1, \dots, g_j \rangle$ for $j = 1, \dots, m$, we have $\{1\} = H_0 \triangleleft H_1 \triangleleft \dots \triangleleft H_m = G$.

Black-box Groups I

Definition (Black-box Groups)

A black-box group G is a group G whose elements are encoded by binary strings of a fixed length n , which is the *encoding length*, and a black-box (or group oracle) performs the group operations at unit cost. The operations include:

- taking a product $g \cdot h$ of elements g and h .
- taking an inverse g^{-1} of the element g .
- deciding whether an element g is the identity.

Remark. The order of a black-box group G of encoding length n is at most 2^n .

In our case, corresponding to a given black-box group G with encoding length n is a unitary operation U_G acting on $2n$ qubits in the following way: $U_G|g\rangle|h\rangle = |g\rangle|gh\rangle$ for any $g, h \in G$. Note that some combinations of the qubits may not give elements in G and

Black-box Groups II

in this case, U_G may act in any arbitrary way as long as the operation is reversible. It is easy to see that the inverse of U_G acts in the following way $U_G^{-1}|g\rangle|h\rangle = |g\rangle|g^{-1}h\rangle$. We will assume that U_G and U_G^{-1} are available for a given black-box group G .

Theorem (Babai&Cooperman, 1995)

With respect to any given group oracle, if we are given generators $g_1, \dots, g_m$ of encoding length n , it is possible to test whether $G = \langle g_1, \dots, g_m \rangle$ is solvable via a polynomial time (in nm) Monte Carlo algorithm. The same algorithm can be used to construct (with high probability) generators $g_1^{(j)}, \dots, g_k^{(j)}$ for $j \in \mathbb{Z}_+$ where $k = O(n)$ such that $G^{(j)} = \langle g_1^{(j)}, \dots, g_k^{(j)} \rangle$.

Remark. If we relabel the elements $g_1^{(n-1)}, \dots, g_k^{(n-1)}, g_1^{(n-2)}, g_k^{(n-2)}, \dots, g_1^{(0)}, \dots, g_k^{(0)}$ as $h_1, \dots, h_{kn}$ and let $H_j = \langle h_1, \dots, h_j \rangle$ for $j = 0, \dots, kn$, then we have the

Black-box Groups III

subnormal series $\{1\} \triangleleft H_0 \triangleleft H_1 \triangleleft \cdots \triangleleft H_{kn} = G$. Note that the quotient group H_j/H_{j-1} is cyclic.

Main Theorem

The algorithm that we are discussing today is proposed by John Watrous in his 2001 paper “Quantum Algorithms for Solvable Groups.”

Theorem (John Watrous, 2001)

There exists a quantum algorithm operating in the following way (relative to an arbitrary group oracle). Given generators $g_1, \dots, g_k$ such that $G = \langle g_1, \dots, g_k \rangle$ is solvable, the algorithm outputs the order of G with probability of error bounded by ϵ in time polynomial in $n + \log(\frac{1}{\epsilon})$ (where n is the length of the strings representing the generators). Moreover, the algorithm produces a quantum state ρ that approximates the pure state $|G\rangle = |G|^{-\frac{1}{2}} \sum_{g \in G} |g\rangle$ with accuracy ϵ (in the trace norm metric).

The quantum algorithm of finding the orders of solvable groups consists of two major parts:

- finding the order of a group element with respect to a subgroup,
- creating uniform superpositions over subgroups.

We assume that we have elements $g_1, \dots, g_m \in G$ such that if we let $H_j = \langle g_1, \dots, g_j \rangle$ where $j = 1, \dots, m$, then $\{1\} = H_0 \triangleleft H_1 \triangleleft \dots \triangleleft H_m = G$ (duplicates allowed). In a previous slide about solvable groups, we discussed that such sequence exists if and only if G is solvable. As is discussed above, given an arbitrary collection of generators of G , such a sequence can be found via a Monte Carlo algorithm in polynomial time. The quotient groups H_j/H_{j-1} are cyclic for $j = 1, \dots, m$. Denote the orders of the quotient groups

$r_1 = |H_1/H_0|, r_2 = |H_2/H_1|, \dots, r_m = |H_m/H_{m-1}|$. By the universal property of quotient groups, we have $|G| = \prod_{j=1}^m r_j$.

Definition (Order with respect to a subgroup)

Given a subgroup $H \leq G$ and an element $g \in G$, we define $r_H(g)$, the order of g with respect to H , to be the smallest positive integer n such that $g^n \in H$ if such n exists.

Since $|H_j/H_{j-1}| = r_j$ for $j = 1, \dots, m$, we can see that the order of g_j with respect to H_{j-1} is r_j . If we can find the order of each element in the generating sequence with respect to the subgroup generated by terms before this element, then the product of these orders would be the order of the whole solvable group.

Only solvable groups guarantee the existence of such generating sequence, non-solvable groups (as simple as S_5) do not necessarily

have such generating sequences. The solvability of the finite group G is necessary in our algorithm.

The order-finding algorithm is a modified algorithm covered in the class to find the order of an element within a multiplicative group modulo m . In order to apply this algorithm, we need to construct the quantum states $|H_0\rangle, \dots, |H_m\rangle$, where

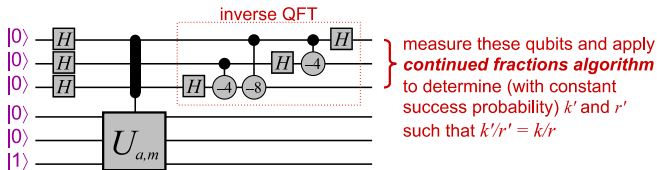
$|H_j\rangle = |H_j|^{-\frac{1}{2}} \sum_{g \in H_j} |g\rangle$ for $j = 1, \dots, m$. The second part of the algorithm would convert $|H_{j-1}\rangle$ into $|H_j\rangle$ for $j = 1, \dots, m$.

In the following discussion, We will write $e_m(k)$ to denote $e^{\frac{2\pi i k}{m}}$ where $m \in \mathbb{Z}_+$ and $k \in \mathbb{Z}_m$ and $|S\rangle = |S|^{-\frac{1}{2}} \sum_{s \in S} |s\rangle$ for any finite set S .

Finding orders with respect to a subgroup I

Recall the quantum algorithm for order finding from class:

Quantum algorithm for order-finding



$$U_{a,m} |y\rangle = |ay \bmod m\rangle$$

Number of gates for $\Omega(1/\log n)$ success probability is: $O(n^2 \log n)$

(this is the cost of $O(n)$ multiplications)

For any **constant** success probability, repeat $O(\log n)$ times and take the smallest resulting r' that satisfies the equation $a^{r'} = 1 \pmod{m}$

First part of the algorithm I

We assume that we are working over a black-box group G with encoding length n , and that a quantum register $\mathbf{R}$ has been initialized to state $|H\rangle$ for some subgroup of G . For a given $g \in G$, we want to find $r = r_H(g)$. Let $\mathbf{A}$ be a quantum register whose basis states correspond to $\mathbb{Z}_N$ for N to be chosen later and we assume $\mathbf{A}$ is initialized to state $|0\rangle$.

- perform the quantum Fourier transform modulo N (F_N) on $\mathbf{A}$;
- reversibly left-multiply the contents of $\mathbf{R}$ by g^a , for a the number contained in $\mathbf{A}$;
- perform the inverse quantum Fourier transform modulo N ($F_N^\dagger$) on $\mathbf{A}$.

First part of the algorithm II

Multiplication by g^a can be easily done reversibly in polynomial time using the group oracle. The state of the pair $(\mathbf{A}, \mathbf{R})$ becomes

$$\frac{1}{N} \sum_{a \in \mathbb{Z}_N} \sum_{b \in \mathbb{Z}_N} e_N(-ab) |b\rangle |g^a H\rangle.$$

Recall the discussion from class:

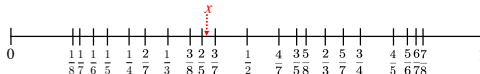
Bypassing the need for $|\psi_1\rangle$ (2)

What if k is chosen randomly and kept secret?

Let $r \in \{1, 2, \dots, m\}$, where m is an n -bit integer and $k \in \{1, 2, \dots, r\}$

Given x , a $2n$ -bit approximation of k/r , can we determine k, r ?

The situation is now more complicated, though in principle we could still imagine checking where x lands on the line and round to the nearest k/r



The hardest case to distinguish is still between $1/m$ vs $1/(m-1)$, where the gap is around $1/m^2$

Of course, we cannot distinguish between these r/k : $1/2 = 2/4 = 3/6 = 4/8$, but at least the procedure makes sense when $\gcd(k, r) = 1$

But: is there an algorithm that finds k and r in time polynomial in n ? 11

First part of the algorithm IV

A measurement of $\mathbf{A}$ yields a value $b \in \mathbb{Z}_N$. With high probability, $\frac{b}{N}$ is a good approximate to $\frac{k}{r}$ (with respect to “modulo 1” distance), where k is randomly distributed in $\mathbb{Z}_r$. Assuming N is sufficiently large, we may find relatively prime integers u and v such that $\frac{u}{v} = \frac{k}{r}$ with high probability by the continued fraction method. Choosing $N = 2^{2n+O(\log(\frac{1}{\epsilon}))}$ allows us to determine u and v with probability $1 - \epsilon$. In order to find r , we repeat this process $O(\log(\frac{1}{\epsilon}))$ times and compute the least common multiple of the v values, which gives r with probability at least $1 - \epsilon$.

Creating uniform superpositions over subgroups I

In the second part of the algorithm, we want to convert several copies of the quantum state $|H\rangle$ to copies of $|\langle g \rangle H\rangle$, where g normalizes H ($gH = Hg$, which ensures that $\langle g \rangle H$ is a group and that $H \triangleleft \langle g \rangle H$). In this part, we assume that $r = r_H(g)$ is known from the first part of the algorithm.

We assume that we have registers $\mathbf{R}_1, \dots, \mathbf{R}_l$, and they are all in state $|H\rangle$. Let $\mathbf{A}_1, \dots, \mathbf{A}_l$ be registers whose basis states corresponds to $\mathbb{Z}_r$, and we assume that $\mathbf{A}_1, \dots, \mathbf{A}_l$ are all initialized to $|0\rangle$. For each $i = 1, \dots, l$, we carry out the following steps:

- perform the quantum Fourier transform modulo r (F_r) on A_i ;
- reversibly left-multiply the contents of $\mathbf{R}_i$ by g^{a_i} where a_i denotes the content of $\mathbf{A}_i$;
- perform the quantum Fourier transform modulo r (F_r) again on A_i .

Creating uniform superpositions over subgroups II

Each pair $(\mathbf{A}_i, \mathbf{R}_i)$ becomes

$$\frac{1}{r} \sum_{a_i \in \mathbb{Z}_r} \sum_{b_i \in \mathbb{Z}_r} e_r(a_i b_i) |b_i\rangle |g^{a_i} H\rangle.$$

Now we measure the registers $\mathbf{A}_1, \dots, \mathbf{A}_l$ and denote the results by $b_1, \dots, b_l$. The state after measurement becomes

$$|\psi_i\rangle = r^{-\frac{1}{2}} \sum_{a_i \in \mathbb{Z}_r} e_r(a_i b_i) |g^{a_i} H\rangle.$$

The probability that none of $b_1, \dots, b_l$ is relatively prime to r is at most ϵ whenever $l \in \Omega((\log \log r)(\log \frac{1}{\epsilon}))$. If there exists b_k where $1 \leq k \leq l$ such that b_k is relatively prime to r , we reversibly multiply the contents of $\mathbf{R}_k$ by f^c , where f is the group element contained in $\mathbf{R}_i$ where $i \neq k$, and $c \in \mathbb{Z}$ satisfies $c \equiv b_i b_k^{-1} \pmod{r}$.

Creating uniform superpositions over subgroups III

We claim that $\mathbf{R}_j$ contains the state $|\langle g \rangle H\rangle$ and $\mathbf{R}_k$ still contains $|\psi_k\rangle$.

Consider a unitary operator $M_{g^j h}$ that left multiplies the contents of $\mathbf{R}_k$ by $g^j h$ (for any $h \in H$). Since g normalizes H , we have

$$\begin{aligned} M_{g^j h} |\psi_k\rangle &= r^{-\frac{1}{2}} \sum_{a_k \in \mathbb{Z}_r} e_r(a_k b_k) |g^{j+a_k} H\rangle \\ &= r^{-\frac{1}{2}} \sum_{a_k \in \mathbb{Z}_r} e_r((a_k - j)b_k) |g^{a_k} H\rangle = e_r(-j b_k) |\psi_k\rangle, \end{aligned}$$

Creating uniform superpositions over subgroups IV

so $|\psi_k\rangle$ is an eigenvector of $M_{g^j h}$ with the eigenvalue $e_r(-jb_k)$. Therefore, after performing the multiplication, the pair $(\mathbf{R}_i, \mathbf{R}_k)$ becomes

$$\begin{aligned} & \frac{1}{\sqrt{r|H|}} \sum_{a_i \in \mathbb{Z}_r} \sum_{h \in H} e_r(a_i b_i) |g^{a_i} h\rangle M_{(g^{a_i} h)^c} |\psi_k\rangle \\ &= \frac{1}{\sqrt{r|H|}} \sum_{a_i \in \mathbb{Z}_r} \sum_{h \in H} e_r(a_i b_i - a_i b_i b_k^{-1} b_k) |g^{a_i} h\rangle |\psi_k\rangle \\ &= \frac{1}{\sqrt{r|H|}} \sum_{a_i \in \mathbb{Z}_r} \sum_{h \in H} |g^{a_i} h\rangle |\psi_k\rangle \\ &= |\langle g \rangle H\rangle |\psi_k\rangle. \end{aligned}$$

Repeat this process for each $i = 1, \dots, l$, where $i \neq k$ and then discard $\mathbf{R}_k$. In this way, we have $l - 1$ copies of $|\langle g \rangle H\rangle$.

Combining two parts of the algorithm I

Assume that we have elements $g_1, \dots, g_m \in G$ such that for $H_j = \langle g_1, \dots, g_j \rangle$ where $j = 1, \dots, m$, we have $\{1\} = H_0 \triangleleft H_1 \triangleleft \dots \triangleleft H_m = G$. Let $r_j = r_{H_{j-1}}(g_j) = |H_j/H_{j-1}|$, where $j = 1, \dots, m$. We combine the two parts of the algorithm in the following way:

- Prepare $k(m+1)$ copies of $|H_0\rangle$, where $H_0 = \{1\}$.
- Iteratively carry out the following steps where the integer j increases from 1 to m :
 - Use $(k-1)$ copies of $|H_{j-1}\rangle$ to compute r_j and discard these states
 - Use one of the copies of $|H_{j-1}\rangle$ to convert the remaining ones to copies of $|H_j\rangle$ and discard this one copy.
- Return the value $\prod_{j=1}^m r_j$, which is the order of G .

Combining two parts of the algorithm II

In order that the algorithm may work correctly with high probability, we must make the error in each step small. If we want the entire process to work with probability of error less than ϵ , we may perform the computations of each of the r_j values such that each computation of r_j fails with probability at most $\frac{\epsilon}{2m}$ and each conversion of $|H_{j-1}\rangle$ to $|H_j\rangle$ where $j = 1, \dots, m$ with error at most $\frac{\epsilon}{2m}$. Therefore, choosing $k = O((\log n)(\log \frac{m}{\epsilon}))$ would suffice.

Further Problems I

We can consider the reverse of the problem, namely, how to find a generating sequence of a finite group G .

Definition (Irredundant Generating Sequence)

Let $g_1, \dots, g_n$ be a generating sequence of a finite group G . If $g_i \notin \langle g_1, \dots, g_{i-1}, g_{i+1}, \dots, g_n \rangle$, where $i = 1, \dots, n$, then we call this generating sequence irredundant.

Unlike vector spaces, whose bases have the same cardinality, irredundant generating sequences of a finite group do not have to have the same length.

Definition

Let $m(G)$ be the maximum length of an irredundant generating sequence of a finite group G and $i(G)$ be the maximum length of any irredundant sequence in G . Let r be the smallest integer n so that G has a generating sequence of size n . It is easy to see that $r(G) \leq m(G) \leq i(G)$.

How do we find the generating sequences of a finite group G ? Or we may pose an “easier” question: how to calculate the dimension-like constants $r(G)$, $m(G)$, $i(G)$?

It turns out that calculating $r(G)$ is fairly easy with a modified Euler's totient function. Calculating $m(G)$ and $i(G)$ are very hard, but there are simple algorithms for SOLVABLE groups. Below is a code in GAP to calculate r for finite groups, and m and i for finite solvable groups.

Further Problems III

```
R:=function(G) local ng,n; r(G) ng:=0; n:=0;
SetInfoLevel(InfoWarning,0); while(ng=0) do n:=n+1;
ng:=EulerianFunction(G,n); od; return n; end;
MSolv:=function(G) local H,N,r; m(G) for solvable G if(not
IsSolvable(G)) then return(infinity); fi; r:=0; H:=G;
while(Size(H)>1) do H:=H/FrattiniSubgroup(H);
N:=MinimalNormalSubgroups(H)[1]; H:=H/N; r:=1+r; od;
return(r); end;
ISolv:=function(G) local r,C,c; i(G) for solvable G if(not
IsSolvable(G)) then return(infinity); fi; seems to be faster r:=0;
C:=ConjugacyClassesSubgroups(G); for c in C do
R:=MSolv(Representative(c)); r:=Maximum(R,r); od; return(r);
end;
ListMaxSg0:=function(arg) local g,max,cmx,y,z,mm,ii;
if(Length(arg)>1) then Print("usage: ListMaxSg(G);"); return; fi;
g:=arg[1]; Print(NiceStructureDescription(g)," ");
```

Further Problems IV

```
cmx:=ConjugacyClassesMaximalSubgroups(g); max:=0; for y in
cmx do max:=max+Size(y); od; Print(max,""); for y in cmx do
z:=Representative(y); mm:=MSolv(z); ii:=ISolv(z); Print("siz =
",RJust(4,Size(y))," ",RJust(9,SafeldGroup(z)),"
",RJust(25,NiceStructureDescription(z))," m = ",mm," i =
",ii,""); od; Print("_____");
end;
end of rmi.gap
```

References



[John Watrous \(2001\).](#)

Quantum algorithms for solvable groups.

[arXiv :quant-ph/0011023v2.](#)



[Daniel Collins \(2017\).](#)

Generating Sequences of Finite Groups.

[Notes.](#)



[Richard Cleve \(2019\).](#)

Introduction to Quantum Information Procession.

[Lecture Notes.](#)

The End

Thank you for listening.